Name of Student: VIKAS FOOLCHAND KUSHWAHA			
Roll Number: 29/B		**Lab Practical Number:** 01	
Title of LAB Practical: Programs based on Java Generics, Collection framework and Lambda expressions.			
DOP: 28/09/2026		**DOS:** 05/10/2026	
CO Mapped: CO1, CO2	**PO Mapped:** PO1, PO2, PO3, PO4, PSO1, PSO2	**Faculty Signature:**	**Marks:**

Practical 1

AIM: Write programs based on Java Generics and Lambda expressions.

Theory:

1. Generics in Java

Generics in Java are used to create classes, methods, and collections that can work with different data types while providing type safety. Generics improve code reusability and reduce the need for type casting.

Syntax:

```
class ClassName<T> {
    T variable;
}
```

Example:

```
class Box<T> {
    T value;
    Box(T value) {
        this.value = value;
    }
}
```

Object creation:

```
Box<Integer> obj = new Box<>(10);
```

---

2. Bounded Types

Bounded types are used to restrict the type that can be passed to a generic class or method. The extends keyword is used to define the bound.

Syntax:

```
class ClassName<T extends Type> {
}
```

Example:

```
class BoundedShape<T extends Shape> {
   T shape;

   BoundedShape(T shape) {
      this.shape = shape;
   }
   void displayArea() {
      System.out.println(shape.area());
   }
}
```

Here, T can only be Shape or a class that implements Shape.

---

3. Interface

An interface is used to define methods that are implemented by classes. It provides abstraction and allows different classes to provide their own implementation of the same method.

Syntax:

```
interface InterfaceName {
   returnType methodName(parameters);
}
```

Example:

```
interface Shape {
   double area();
}
```

A class can implement the interface using the implements keyword.

Syntax:

```
class ClassName implements InterfaceName {
```

```
    // method implementation
}
```

Example:

```
class Circle implements Shape {
    public double area() {
        return Math.PI * 5 * 5;
    }
}
```

---

4. Generic Class with Number

A generic class can be restricted to numeric data types by using the Number class as a bound. This allows classes such as Integer, Double, Float, and other numeric wrapper classes to be used.

Syntax:

```
class ClassName<T extends Number> {
    T[] array;
}
```

Example:

```
class GenericArray<T extends Number> {
    T[] array;
    GenericArray(T[] array) {
        this.array = array;
    }
    double average() {
        double sum = 0;
        for (T value : array) {
            sum += value.doubleValue();
        }
```

```
        return sum / array.length;
    }
}
```

Here, doubleValue() converts the numeric value into double so that arithmetic calculations can be performed.

---

5. Wildcard in Generics

A wildcard is used when the exact generic type is not known. The extends keyword can be used with a wildcard to accept any subclass of a particular type.

Syntax:

ClassName<? extends Type>

Example:

GenericArray<? extends Number> obj;

This allows a GenericArray containing Integer, Double, or another subclass of Number.

---

6. Lambda Expression

A lambda expression is a short way of providing an implementation for a functional interface. It makes the code more concise and is commonly used for simple operations.

Syntax:

(parameters) -> expression

Example:

(a, b) -> a + b

Arithmetic operations can be written as:

(a, b) -> a + b

(a, b) -> a - b

(a, b) -> a * b

(a, b) -> a / b

---

7. Functional Interface

A functional interface is an interface that contains exactly one abstract method. It can be implemented using a lambda expression.

Syntax:

```
@FunctionalInterface
interface InterfaceName {
   returnType methodName(parameters);
}
```

Example:

```
@FunctionalInterface
interface Operation {
   double calculate(double a, double b);
}
```

The interface can be implemented using a lambda expression:

```
Operation add = (a, b) -> a + b;
```

---

8. Collection Framework

The Java Collection Framework provides classes and interfaces for storing and manipulating groups of objects. It includes collections such as ArrayList, LinkedList, and HashMap.

These collections provide methods for adding, removing, accessing, searching, and traversing elements.

---

9. ArrayList

ArrayList is a resizable collection that stores elements in insertion order. Its size can change dynamically when elements are added or removed. Generics can be used to specify the type of elements stored in the list.

Syntax:

```
ArrayList<Type> list = new ArrayList<>();
```

Example:

```
ArrayList<Integer> numbers = new ArrayList<>();

numbers.add(10);

numbers.add(20);

numbers.add(30);
```

Traversing the ArrayList:

```
for (Integer number : numbers) {
    System.out.println(number);
}
```

Common methods:

```
list.add(element);

list.get(index);

list.remove(index);

list.size();
```

---

10. LinkedList

LinkedList is a collection in which elements are stored as linked nodes. It supports adding, removing, and traversing elements. A LinkedList can be traversed in both forward and backward directions.

Syntax:

```
LinkedList<Type> list = new LinkedList<>();
```

Example:

```
LinkedList<String> list = new LinkedList<>();

list.add("Apple");

list.add("Banana");

list.add("Mango");

list.add("Orange");

list.add("Grapes");
```

Forward traversal:

```
Iterator<String> it = list.iterator();
while (it.hasNext()) {
   System.out.println(it.next());
}
```

Backward traversal:

```
Iterator<String> it = list.descendingIterator();
while (it.hasNext()) {
   System.out.println(it.next());
}
```

---

11. Iterator

An Iterator is used to traverse elements of a collection one by one. It provides methods such as hasNext() and next() to access elements.

Syntax:

```
Iterator<Type> iterator = collection.iterator();
while (iterator.hasNext()) {
   Type element = iterator.next();
}
```

Example:

```
Iterator<String> it = list.iterator();
while (it.hasNext()) {
   System.out.println(it.next());
}
```

For backward traversal of a LinkedList:

```
Iterator<String> it = list.descendingIterator();
```

Question 1. Implement bounded types (implements an interface) with generics.

Create an Interface shape with method Area() create Circle and Square which implements Class Shape.

Create a generic class BoundedShape that extends shape. And implement the generics and use area function accordingly.

CODE

```
 interface Shape {
      double area();
}

class Circle implements Shape {
      private double radius;

      Circle(double radius) {
            this.radius = radius;
      }

      @Override
      public double area() {
            return Math.PI * radius * radius;
      }
}

class Square implements Shape {
      private double side;

      Square(double side) {
            this.side = side;
      }

      @Override
      public double area() {
            return side * side;
      }
}

class BoundedShape<T extends Shape> {
      private T shape;
```

CODE

```
      BoundedShape(T shape) {
            this.shape = shape;
      }

      public void displayArea() {
            System.out.println("Area = " + shape.area());
      }
}

public class Question1 {
      public static void main(String[] args) {
            Circle c = new Circle(7);
            Square s = new Square(25);

            BoundedShape<Circle> circleShape = new BoundedShape<>(c);
            BoundedShape<Square> squareShape = new BoundedShape<>(s);

            circleShape.displayArea();
            squareShape.displayArea();
      }
}
```

OUTPUT:

```
> javac Question1.java && java Question1
Area = 153.93804002589985
Area = 625.0
```

Question 2: Create Generic class , contains array of generic type, write a method to calculate average of the members of the array, write a method to check whether the average of array of different object is equal or not.

CODE

```
  class GenericArray<T extends Number> {
```

CODE

```
	private T[] array;

	GenericArray(T[] array) {
		this.array = array;
	}

	public double average() {
		double sum = 0;

		for (T value : array)
			sum += value.doubleValue();

		return sum / array.length;
	}

	public boolean isAverageEqual(GenericArray<? extends Number>
other) {
		return this.average() == other.average();
	}
}

public class Question2 {

	public static void main(String[] args) {
		Integer[] a = { 40, 20, 30, 40, 20 };
		Double[] b = { 60.0, 10.0, 20.0 };

		GenericArray<Integer> obj1 = new GenericArray<>(a);
		GenericArray<Double> obj2 = new GenericArray<>(b);

		System.out.println("Average of Integer array = " +
obj1.average());
		System.out.println("Average of Double array = " +
obj2.average());

		if (obj1.isAverageEqual(obj2))
			System.out.println("Average is equal");

		else
			System.out.println("Average is not equal");
	}
}
```

OUTPUT:

```
> javac Question2.java && java Question2
Average of Integer array = 30.0
Average of Double array = 30.0
Average is equal
```

Question 3: Perform addition, subtraction, multiplication as well as division using Lambda Expression

CODE

```
@FunctionalInterface
interface Operation {
	double calculate(double a, double b);
}

public class Question3 {
	public static void main(String[] args) {
		Operation addition = (a, b) -> a + b;
		Operation subtraction = (a, b) -> a - b;
		Operation multiplication = (a, b) -> a * b;
		Operation division = (a, b) -> a / b;
		double a = 20;
		double b = 10;
		System.out.println("Addition = " + addition.calculate(a,
b));
		System.out.println("Subtraction = " +
subtraction.calculate(a, b));
		System.out.println("Multiplication = " +
multiplication.calculate(a, b));
		System.out.println("Division = " + division.calculate(a,
b));
	}
}
```

OUTPUT:

```
> javac Question3.java && java Question3
Addition = 30.0
Subtraction = 10.0
Multiplication = 200.0
Division = 2.0
```

Question 4: Create an ArrayList of type Integer, add element into it traverse the arraylist and print the elements

CODE

```
import java.util.ArrayList;

public class Question4 {
  public static void main(String[] args) {
    ArrayList<Integer> numbers = new ArrayList<>();

    numbers.add(1);
    numbers.add(2);
    numbers.add(3);
    numbers.add(4);
    numbers.add(5);

    System.out.println("ArrayList elements : ");

    for (Integer number : numbers)
      System.out.println(number);
  }
}
```

OUTPUT:

```
> javac Question4.java && java Question4
ArrayList elements :
1
2
3
4
5
```

Question 5: Create a LinkedList of type String add 5 elements and traverse the list and from both sides

CODE

```
 import java.util.Iterator;
import java.util.LinkedList;

public class Question5 {
  public static void main(String[] args) {
    LinkedList<String> list = new LinkedList<>();

    list.add("Vikas");
    list.add("Ramesh");
    list.add("Guddi");
    list.add("Abhishek");
    list.add("Mandeep");

    System.out.println("Forward Traversal : ");

    Iterator<String> forward = list.iterator();
    while (forward.hasNext())
      System.out.println(forward.next());

    System.out.println("\nBackward Traversal : ");

    Iterator<String> backward = list.descendingIterator();
    while (backward.hasNext())
      System.out.println(backward.next());
  }
```

CODE

```
}
```

OUTPUT:

```
> javac Question5.java && java Question5
Forward Traversal :
Vikas
Ramesh
Guddi
Abhishek
Mandeep

Backward Traversal :
Mandeep
Abhishek
Guddi
Ramesh
Vikas
```

Question 6: Create an employee class (id, name, salary) create an Arralist of type employee, add 5 employee, traverse the ArrayList and print the elements, Remove one element and print the list

CODE

```
import java.util.ArrayList;

class Employee {
  int id;
  String name;
  double salary;

  Employee(int id, String name, double salary) {
```

CODE

```
    this.id = id;
    this.name = name;
    this.salary = salary;
  }

  public String toString() {
    return "ID : " + id + ", Name : " + name + ", Salary : " +
salary;
  }
}

public class Question6 {

  public static void main(String[] args) {
    ArrayList<Employee> employees = new ArrayList<>();

    employees.add(new Employee(101, "Vikas", 30000));
    employees.add(new Employee(102, "Guddi", 35000));
    employees.add(new Employee(103, "Mandeep", 45000));
    employees.add(new Employee(104, "Abhishek", 40000));
    employees.add(new Employee(105, "Ramesh", 50000));

    System.out.println("Employee List : ");

    for (Employee e : employees)
      System.out.println(e);

    employees.remove(3);

    System.out.println("\nAfter removing one employee : ");

    for (Employee e : employees)
      System.out.println(e);
  }
}
```

OUTPUT:

```
> javac Question6.java && java Question6
Employee List :
ID : 101, Name : Vikas, Salary : 30000.0
ID : 102, Name : Guddi, Salary : 35000.0
ID : 103, Name : Mandeep, Salary : 45000.0
ID : 104, Name : Abhishek, Salary : 40000.0
ID : 105, Name : Ramesh, Salary : 50000.0

After removing one employee :
ID : 101, Name : Vikas, Salary : 30000.0
ID : 102, Name : Guddi, Salary : 35000.0
ID : 103, Name : Mandeep, Salary : 45000.0
ID : 105, Name : Ramesh, Salary : 50000.0
```

Question 7: Create a class Customer(Account_no Integer, Name Sting), Create a HashMap of type Customer put elements, print elements, check if element with account number 101 is present or not? What is the value for Customer 101.

CODE

```
import java.util.HashMap;

class Customer {
  int account_no;
  String name;

  Customer(int account_no, String name) {
    this.account_no = account_no;
    this.name = name;
  }

  public String toString() {
    return "Account No : " + account_no + ", Name : " + name;
  }
}
```

CODE

```
public class Question7 {
  public static void main(String[] args) {
    HashMap<Integer, Customer> customers = new HashMap<>();

    customers.put(101, new Customer(101, "Vikas"));
    customers.put(102, new Customer(102, "Abhishek"));
    customers.put(103, new Customer(103, "Mandeep"));
    customers.put(104, new Customer(104, "Ramesh"));
    customers.put(105, new Customer(105, "Guddi"));
    System.out.println("Customer Details : ");

    for (Customer c : customers.values())
      System.out.println(c);

    if (customers.containsKey(101)) {
      System.out.println("\nAccount number 101 is present.");
      System.out.println("Customer " + customers.get(101));
    } else
      System.out.println("\nAccount number 101 is not present.");
  }
}
```

OUTPUT:

```
> javac Question7.java && java Question7
Customer Details :
Account No : 101, Name : Vikas
Account No : 102, Name : Abhishek
Account No : 103, Name : Mandeep
Account No : 104, Name : Ramesh
Account No : 105, Name : Guddi

Account number 101 is present.
Customer Account No : 101, Name : Vikas
```

CONCLUSION:

1. **Bounded Generics and Interface:** We successfully implemented the Shape interface using Circle and Square classes and used bounded generics to restrict the generic type to classes implementing Shape. This demonstrates the use of interfaces, generics, and polymorphism in Java.
2. **Generic Class:** We created a generic class containing an array of numeric values and calculated the average of its elements. We also compared the averages of arrays of different numeric types. This demonstrates the use of generic classes, bounded types, and reusable code.
3. **Lambda Expression:** We successfully performed addition, subtraction, multiplication, and division using lambda expressions. This demonstrates the use of functional interfaces and concise implementation of operations in Java.
4. **ArrayList:** We created an ArrayList of integers, added elements, traversed the list, and displayed the elements. This demonstrates the use of the ArrayList collection and dynamic storage of elements.
5. **LinkedList:** We created a LinkedList of strings, added five elements, and traversed the list in both forward and backward directions. This demonstrates the use of LinkedList and iterators for bidirectional traversal.
6. **Employee ArrayList:** We created an Employee class and stored five employee objects in an ArrayList. We traversed the list, displayed the employee details, and removed an element. This demonstrates the use of user-defined objects with collections.
7. **HashMap:** We created a Customer class and stored customer objects in a HashMap using account numbers as keys. We displayed the elements and checked whether account number 101 was present and retrieved its corresponding customer. This demonstrates the use of key-value pairs and important HashMap methods such as put(), get(), and containsKey().

Overall, these programs demonstrate the practical use of **Generics, Interfaces, Lambda Expressions, ArrayList, LinkedList, Iterator and HashMap** in Java.